Examen Unidad 2: Visión Artificial.

Adrián Andrés Gallardo Esqueda.

Este documento será el examen practico de la segunda unidad de la materia de visión artificial lo que se busca hacer en este examen es en base a 15 imágenes de dados a través de un código contar el numero de dados contenidos en la imagen, así como de contar el numero de puntos que tiene cada dado en la imagen. Todo esto haciendo uso de las funciones ya vistas en clase acerca de preprocesamiento de imágenes y conectividad de componentes.

Código propuesto:

```
import cv2
import numpy as np
import os

# PREPROCESAMIENTO
def preprocesar_imagen(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8,8))
    gray = clahe.apply(gray)
    blur = cv2.GaussianBlur(gray, (5,5), 0)
    thresh = cv2.adaptiveThreshold(
        blur, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        11, 2
    )

    kernel = np.ones((3,3), np.uint8)
    morph = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=2)
    return morph

# DETECCIÓN DE DADOS

def detectar_dados(binaria, area_min=3000, area_max=15000):
    num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(binaria)
    dados = []
    for i in range(1, num_labels):
        area = stats[i, cv2.CC_STAT_AREA]
        x = stats[i, cv2.CC_STAT_LEFT]
        y = stats[i, cv2.CC_STAT_TOP]
        w = stats[i, cv2.CC_STAT_WIDTH]
        h = stats[i, cv2.CC_STAT_HEIGHT]

        aspect_ratio = w / h
        if (area_min < area < area_max and
            0.4 < aspect_ratio < 1.3 and
            w > 20 and h > 20):
            dados.append((x, y, w, h))
    return dados

# CONTEO DE PUNTOS
def contar_puntos(dado_img):
```

```
    gray = cv2.cvtColor(dado_img, cv2.COLOR_BGR2GRAY)
    thresh = cv2.adaptiveThreshold(
        gray, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        11, 2
    )
    kernel = np.ones((3,3), np.uint8)
    thresh = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

    contornos, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    puntos = 0
    for cnt in contornos:
        area = cv2.contourArea(cnt)
        perimetro = cv2.arcLength(cnt, True)
        if perimetro == 0:
            continue
        circularidad = 4 * np.pi * area / (perimetro * perimetro)
        if 25 < area < 300 and 0.6 < circularidad < 1.2:
            puntos += 1
    return puntos

# LEER ANNOTATIONS (YOLO)
def leer_valores_reales(ruta_label):
    valores = []
    with open(ruta_label, "r") as f:
        for linea in f.readlines():
            partes = linea.strip().split()
            if len(partes) > 0:
                valor = int(partes[0])  # clase
                valores.append(valor)
    #  ajuste automático (si usa clases 0–5)
    if len(valores) > 0 and min(valores) == 0:
        valores = [v + 1 for v in valores]
    return valores

# PROCESAR CARPETA
def procesar_carpeta(carpeta_img, carpeta_labels=None):
    error_dados = []
    error_valores = []

    for archivo in os.listdir(carpeta_img):
        if archivo.endswith(".png") or archivo.endswith(".jpg"):

print("\n==========================")
            print("Imagen:", archivo)
            ruta = os.path.join(carpeta_img, archivo)
            img = cv2.imread(ruta)
            if img is None:
                print("Error cargando imagen")
                continue
            binaria = preprocesar_imagen(img)
            dados = detectar_dados(binaria)
            valores_detectados = []
            for (x, y, w, h) in dados:
                dado_roi = img[y:y+h, x:x+w]
```

```
            valor = contar_puntos(dado_roi)
            valores_detectados.append(valor)
        print("Dados detectados:", len(dados))
        print("Valores detectados:", valores_detectados)

        # VALORES REALES
        if carpeta_labels:
            nombre_txt = os.path.splitext(archivo)[0] + ".txt"
            ruta_label = os.path.join(carpeta_labels, nombre_txt)
            if os.path.exists(ruta_label):
                valores_reales = leer_valores_reales(ruta_label)
                print("Dados reales:", len(valores_reales))
                print("Valores reales:", valores_reales)

                # ERROR DADOS
                e_d = abs(len(dados) - len(valores_reales))
                error_dados.append(e_d)

                # ERROR VALORES
                vd = sorted(valores_detectados)
                vr = sorted(valores_reales)
                n = min(len(vd), len(vr))
                if n > 0:
                    diff = sum(abs(vd[i] - vr[i]) for i in range(n)) / n
                    error_valores.append(diff)

    # RESULTADOS FINALES

print("\n==========================")
    print("RESULTADOS FINALES")
    if error_dados:
        print("Error medio (dados):", np.mean(error_dados))
    else:
        print("Error medio (dados): No calculado")
    if error_valores:
        print("Error medio (valores):", np.mean(error_valores))
    else:
        print("Error medio (valores): No calculado")

# EJECUCIÓN
carpeta_imagenes = "imgexamenu2"
carpeta_annotations = "Annotations"

procesar_carpeta(carpeta_imagenes, carpeta_annotations)
```

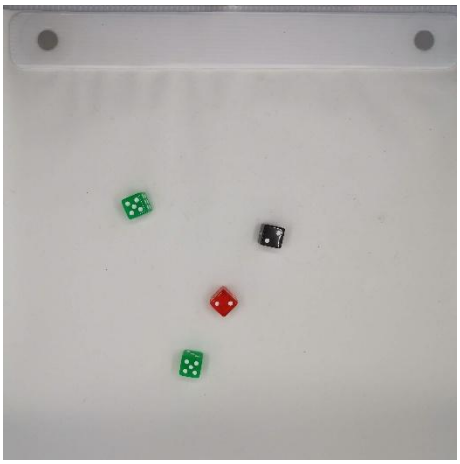
imágenes a procesar:

IMG_20191209_100614

IMG_20191209_100620

IMG_20191209_100625

IMG_20191209_100629

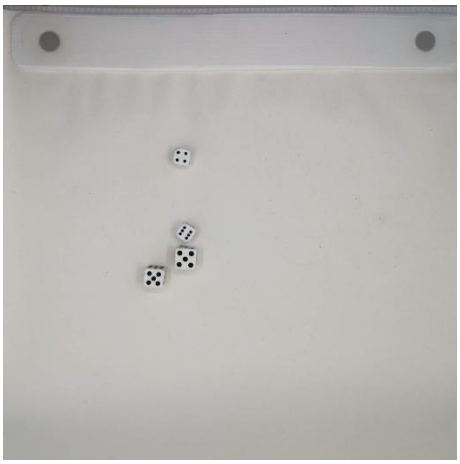


IMG_20191209_100633

IMG_20191209_100641

IMG_20191209_100645

IMG_20191209_100649



IMG_20191209_100714

IMG_20191209_100701

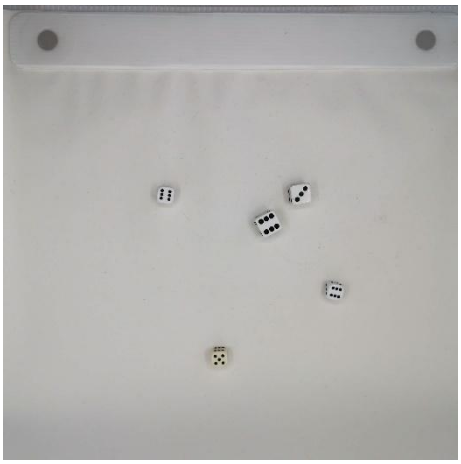
IMG_20191209_100719

IMG_20191209_100706

IMG_20191209_100722

IMG_20191209_100728

IMG_20191209_100733



A continuación, se muestra una tabla de la comparación entre los datos obtenidos con el programa y los datos reales de las imágenes respecto al número de dados en las imágenes y el número de puntos en cada dado de cada imagen:

Imagen	**Dados detectados**	**Dados reales**	**Valores detectados**	**Valores real**
IMG_20191209_100614.jpg	3	3	[3, 0, 1]	[2, 1, 3]
IMG_20191209_100620.jpg	4	4	[1, 5, 3, 4]	[1, 3, 3, 6]
IMG_20191209_100625.jpg	4	4	[4, 8, 2, 4]	[1, 2, 3, 3]
IMG_20191209_100629.jpg	4	4	[3, 3, 1, 5]	[1, 1, 4, 4]
IMG_20191209_100633.jpg	4	4	[5, 7, 0, 4]	[1, 2, 2, 3]
IMG_20191209_100641.jpg	6	6	[2, 5, 0, 1, 8, 0]	[1, 1, 2, 2, 5,
IMG_20191209_100645.jpg	6	6	[7, 7, 1, 2, 6, 3]	[1, 2, 2, 3, 4,
IMG_20191209_100649.jpg	6	6	[1, 5, 1, 5, 2, 8]	[1, 1, 1, 3, 4,
IMG_20191209_100701.jpg	3	4	[1, 0, 1]	[1, 4, 4, 2]
IMG_20191209_100706.jpg	1	4	[7]	[3, 5, 4, 4]
IMG_20191209_100714.jpg	3	5	[2, 0, 1]	[1, 1, 2, 2, 2]
IMG_20191209_100719.jpg	4	5	[0, 0, 0, 2]	[1, 3, 4, 4, 4]
IMG_20191209_100722.jpg	4	5	[0, 0, 0, 4]	[3, 3, 3, 5, 5]
IMG_20191209_100728.jpg	3	5	[0, 2, 0]	[1, 2, 3, 3, 5]
IMG_20191209_100733.jpg	2	5	[0, 1]	[5, 5, 5, 4, 2]

Resultados finales (error).

Error medio (dados): 0.867

Error medio (valores): 1.672

Link de repositorio de GitHub:

https://github.com/adriangallardo9/vision_artificial